Paxos Overview

the original consensus algorithm, in plain words

Watch the 4-min video

In one line: Paxos makes a majority of acceptors agree on one value in two phases (prepare/promise, then accept/accepted) using increasing proposal numbers, and a proposer must adopt any value already accepted; Multi-Paxos runs it per log slot with a stable leader, and is what Raft simplified.

1 Why learn Paxos

Before Raft there was **Paxos**. Leslie Lamport described it in the late 1980s, and it became the foundation of Google's Chubby, Spanner and Megastore. It's also famous for being hard to understand. You rarely implement it yourself, but interviews and papers refer to it constantly, and it shows the core idea every consensus algorithm uses.

2 Three roles



Figure 1. The roles in Basic Paxos. In practice one server plays all three.

3 Basic Paxos: agreeing on one value

Phase 1, prepare: a proposer picks a proposal number *n* higher than any it has used and sends `prepare(n)` to the acceptors. An acceptor that hasn't promised a higher number replies with a **promise** never to accept proposals below *n*, and reports the highest-numbered value it has already accepted, if any.

Phase 2, accept: with promises from a majority, the proposer sends `accept(n, v)`. The key rule: **v must be the value from the highest-numbered accepted proposal reported**, and only if none was reported may it choose its own value. An acceptor accepts unless it has since promised a higher number. When a majority accepts, the value is **chosen**, permanently.

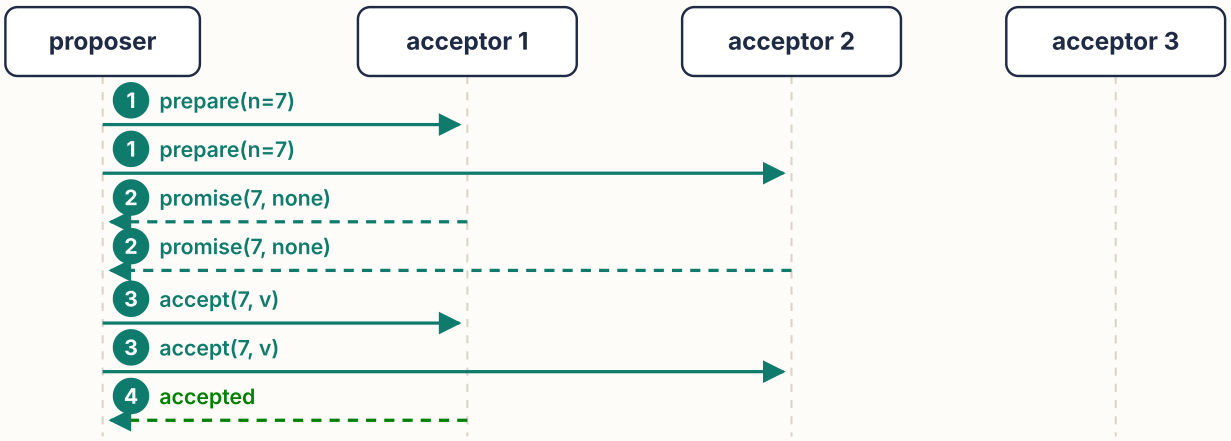


Figure 2. Basic Paxos with a majority of 2 out of 3 acceptors. The third acceptor can be down.

Why it's safe: any two majorities overlap in at least one acceptor. A later proposer's prepare reaches at least one acceptor that accepted the chosen value, learns about it, and has to propose that same value. Why it might not finish: two proposers can keep pre-empting each other with higher numbers (**livelock**); electing a single distinguished proposer avoids that.

4 Multi-Paxos

Real systems need a log, not one value. **Multi-Paxos** runs a Paxos instance per log slot, and lets a stable leader run phase 1 once for many slots, so each new command needs just one round trip. That's effectively what Raft spells out in detail.

WORKED EXAMPLE: GOOGLE'S CHUBBY

Chubby is a lock service with five replicas. They use Multi-Paxos to elect a master and replicate a log of operations. Clients find the master and send it all reads and writes; each write is committed once three of five replicas accept it. The master holds a lease, so it can serve reads locally without a round trip. If the master fails, the others run Paxos to choose a new one, typically within seconds. Many Google systems, from GFS to Bigtable, used Chubby for leader election and configuration.

	Paxos	Raft
decides	one value (Multi-Paxos: a log)	a log, from the start
leader	optional, an optimisation	required, strong leader
spec	short paper, many gaps	full algorithm, easier to build
used in	Chubby, Spanner, Megastore	etcd, Consul, CockroachDB

Figure 3. Paxos and Raft give the same guarantees; Raft fixes the parts Paxos leaves open.

COMMON MISTAKES

- Thinking a value is chosen when one acceptor accepts it: it needs a majority.
- Letting a proposer ignore previously accepted values.
- Building your own Paxos instead of using etcd, ZooKeeper or a proven library.

5 Key terms

Proposer / acceptor / learner: the roles in Paxos

Proposal number: unique, increasing ballot ID

Promise: acceptor won't accept lower-numbered proposals

Chosen: accepted by a majority; final

Multi-Paxos: Paxos per log slot with a stable leader

Livelock: competing proposers block progress

6 Interview questions

Q1. What are the two phases of Paxos? BASIC

Prepare/promise, where a proposer reserves a proposal number and learns any accepted values; then accept/accepted, where a majority accepts the value.

Q2. Why does Paxos need a majority? BASIC

Any two majorities share at least one acceptor, so a later proposal always learns about a value that may already be chosen, which keeps decisions consistent.

Q3. Two proposers keep overriding each other and nothing is decided. Why, and what's the fix? SCENARIO

Duelling proposers keep issuing higher numbers; use a single leader (distinguished proposer) and random back-off.

Q4. Paxos or Raft for a new system? SCENARIO

Neither by hand: use etcd, ZooKeeper or a library. If you must implement, Raft is easier to get right because it specifies leader election, log replication and membership changes.

Q5. Why must a proposer adopt the highest-numbered accepted value? TRICKY

That value may already be chosen by some majority; proposing anything else could make two different values chosen.

Q6. How does Multi-Paxos reduce latency? TRICKY

A stable leader runs phase 1 once for many slots, so each command needs only the accept phase: one round trip to a majority.

Next: SD-H-011 · Quorums backendroadmap.com